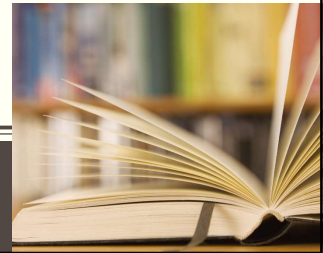
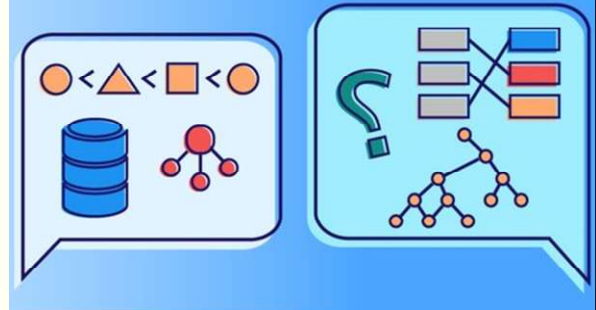


ALGORITMOS Y ESTRUCTURAS DE DATOS

Comisión F – 2026 – Cursado Anual



1

Algoritmos y Estructuras de Datos – AEDD – Agenda 24/08

- Arreglos Multidimensionales.
- Acceso a los elementos de la estructura.
- Inicialización de arreglos bidimensionales.
- Las estructuras como elemento de arreglos.
- Acceso a los elementos de la matriz.
- Estructuras que contiene arreglos.
- Pasaje de matriz como parámetro
- Estructuras como parámetros de funciones.
- Definición de Estructura.
- Ejemplo del pasaje por valor y por referencia.
- Declaración de variables de estructura.
- Función que retorna Struct
- Inicialización de variables de estructura.

2

Contraseña del estudiante

jip5vz

3

Estructuras de datos

- **Simples o básicos:** caracteres, reales, flotantes.
- **Estructurados:** colección de valores relacionados. Se caracterizan por el tipo de dato de sus elementos, la forma de almacenamiento y la forma de acceso.
 - **Estructuras estáticas:** Su tamaño en memoria se mantiene inalterable durante la ejecución del programa, y ocupan posiciones fijas.
ARREGLOS - CADENAS - ESTRUCTURAS
 - **Estructuras dinámicas:** Su tamaño varía durante el programa y no ocupan posiciones fijas.
LISTAS - PILAS - COLAS - ARBOLES - GRAFOS

4

Estructuras de datos - Clasificaciones

- Según donde se almacenan
 - Internas (en memoria principal)
 - Externas (en memoria auxiliar)
- Según tipos de datos de sus componentes
 - Homogéneas (todas del mismo tipo)
 - No homogéneas (pueden ser de distinto tipo)
- Según la implementación
 - Provistas por los lenguajes (básicas)
 - Abstractas (TDA - Tipo de dato abstracto que puede implementarse de diferentes formas)
- Según la forma de almacenamiento
 - Estáticas (ocupan posiciones fijas y su tamaño nunca varía durante todo el módulo)
 - Dinámicas (su tamaño varía durante el módulo y sus posiciones también)

5

Arreglos Multidimensionales

- Son arreglos que permiten varios índices.
- A los arreglos bidimensionales se los llama comúnmente matrices, tablas o arreglos de arreglos.
- A los de 3 dimensiones, se los denomina cubos
- Y al resto, en general, se los denomina multidimensionales.

Declaración de Arreglos Bidimensionales:

Formato de declaración:

tipo_dato nombre-del-arreglo [nroDeFilas][nroDeColumnas];

- Ejemplo: Definir un arreglo bidimensional de 5 x 3 enteros:

int Matriz [5][3];

6

Arreglos Multidimensionales

- Declaración utilizando tipo estructurado:
- Formato de declaración:

typedef tipo_dato nombre-del-arreglo [dim1][dim2];

- Ejemplo: Definir un arreglo bidimensional de 5 x 3 enteros:

typedef int tMatriz [5][3];

- Creación de variables:

Formato de declaración:

tipo_dato nombre-variable;

Ejemplo: tMatriz m1;

7

Arreglos Multidimensionales

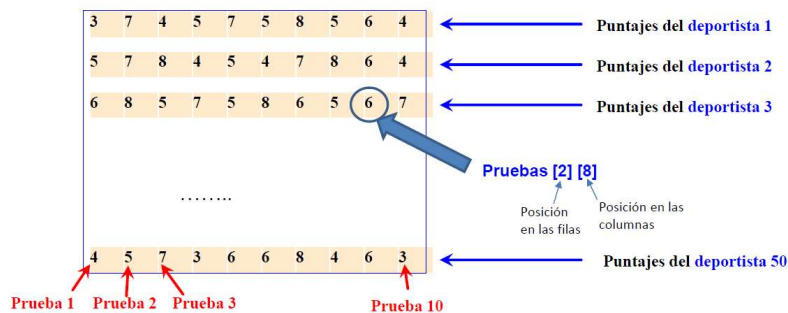
EJEMPLO 1:

- Si se deben almacenar los puntajes de un deportista en 10 pruebas:

int Pruebas [10]; ← Arreglo

- Si se deben almacenar los puntajes de 50 deportistas en 10 pruebas:

int Pruebas [50] [10]; ← Matriz

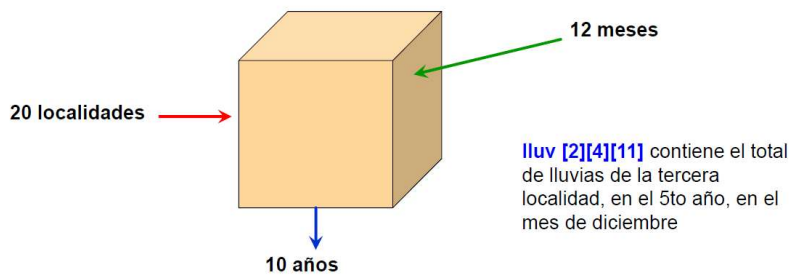


8

Arreglos Multidimensionales

- Las dimensiones pueden ser múltiples.
- Si se desea almacenar los totales mensuales de lluvias mensuales en 20 localidades, durante cada uno de los meses de una década:

```
float lluv [20] [10] [12];
```



C++ proporciona la posibilidad de almacenar varias dimensiones, aunque raramente los datos del mundo real requieren más de dos o tres dimensiones.

9

Arreglos Multidimensionales

- Se pueden definir arreglos con componentes de otros arreglos previamente definidos.
- Ejemplos:


```
typedef char tAlumno[15]; // 15 caracteres
typedef tAlumno tClase[85]; // 85 componentes del tipo tAlumno
typedef int tDatos[3]; // 3 enteros
typedef tDatos tDatosSemanales[5]; // 5 componentes del tipo tDatos
tDatosSemanales datSem = { {9,13,10}, {10,4,8}, {8,19,7}, {16,21,5}, {18,20,19} };
```
- Se pueden definir matrices (tablas) como constantes, de la misma manera que se hace con los tipos elementales.
- Ejemplos:


```
const int mat[3][2] = { {0,2}, {4,4}, {10,10} };
const int mat[ ][2] = { {0,2}, {4,4}, {10,10} };
```

10

Pasaje de arreglos bidimensionales como parámetros

```
#include <iostream>
#include <stdlib.h>
#include <time.h>
#include <iomanip>
using namespace std;

const int DIM1=10;
const int DIM2=10;

void cargarMatriz(int m[][DIM2], int cf, int cc);
void mostrarMatriz(int m[][DIM2], int cf, int cc);

int main() {
    int matriz[DIM1][DIM2];
    int cf=4, cc=3;
    cargarMatriz(matriz, cf, cc);
    mostrarMatriz(matriz, cf, cc);
    return 0;
}
```

```
void cargarMatriz(int m[][DIM2], int cf, int cc){
    //carga la matriz por filas
    int f, c;
    srand(time(NULL));
    for (f=0; f<cf; f++)
        for (c=0; c<cc; c++)
            m[f][c] = rand()%10+1;
}
```

Si se inserta un número dentro de los corchetes el compilador lo ignorará

```
void mostrarMatriz(int m[][DIM2], int cf, int cc){
    int f, c;
    cout << endl;
    for (f=0; f<cf; f++){
        for (c=0; c<cc; c++){
            cout << setw(2) << m[f][c] << " ";
        }
        cout << endl;
    }
}
```

10	2	6
10	8	10
3	1	8
1	1	5

11

Algoritmos y Estructuras de Datos – AEDD – Asistencia 24/08

Contraseña del estudiante
jip5vz

12

Estructuras de datos: Clasificaciones

- Según donde se almacenan
 - Internas (en memoria principal)
 - Externas (en memoria auxiliar)
- Según tipos de datos de sus componentes
 - Homogéneas (todas del mismo tipo)
 - No homogéneas (pueden ser de distinto tipo)
- Según la implementación
 - Provistas por los lenguajes (básicas)
 - Abstractas (TDA - Tipo de dato abstracto que puede implementarse de diferentes formas)
- Según la forma de almacenamiento
 - Estáticas (ocupan posiciones fijas y su tamaño nunca varía durante todo el módulo)
 - Dinámicas (su tamaño varía durante el módulo y sus posiciones también)

13

Estructuras (Struct o Registros)

- Colección de variables relacionadas, bajo un mismo nombre
- Pueden contener variables de diferentes tipos de datos
- Se usa habitualmente para definir registros que van a ser almacenados en archivos o en arreglos.
- Combinada con punteros, pueden crear listas enlazadas, pilas, colas y árboles.

14

Definición de Estructura

Ejemplo 1:

```
struct Carta {
    int valor;
    string palo; };
```

- struct introduce la definición para la estructura Carta
- Carta es el nombre de la estructura y se usa para declarar variables de ese tipo estructura
- La estructura Carta contiene dos campos, de distinto tipo: Estos campos son valor y palo.

La definición de la estructura Carta no reserva espacio en memoria; solo crea un nuevo tipo de dato que puede ser usado para declarar variables de estructura.

15

Definición de Estructura

Ejemplo 2:

```
struct Fecha {
    unsigned dia;
    unsigned mes;
    unsigned anio; };
```

- Los identificadores dia, mes y anio representan los nombres de sus elementos componentes, denominados campos.
- El ámbito de visibilidad de los campos de una estructura se restringe a la propia definición de la estructura o registro.
- Los campos de un registro pueden ser de cualquier tipo de datos, simple o compuesto.

16

Declaración de Variables de estructura

Se puede utilizar una lista de variables separadas por comas, luego de la llave de cierre:

```
struct Carta {
    int valor;
    string palo; } cartamia, mazo[52];
```

O se pueden declarar variables, listándolas en cualquier parte del programa antes de utilizarlas:

```
Carta cartamia, mazo[52];
```

Se reserva espacio de almacenamiento para dos variables de la estructura Carta llamadas cartamia y mazo

17

Inicialización de variables de estructura

- Mediante listas inicializadoras:

Ejemplo: Carta carta1 = { 8, "Trebol" };

- Mediante sentencias de asignación:

Ejemplo: Carta carta2 = carta1;

- Accediendo a sus campos mediante el operador punto (.):

```
Carta carta1;
carta1.valor = 8;
carta1.palo = "Trebol";
```

El operador punto (.) se usa asociado al nombre de la variable (y no al nombre del struct).

18

Ejemplo: Creación de una variable de estructura

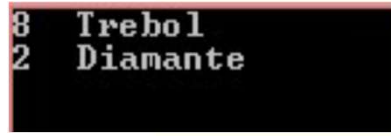
```
#include <iostream>
#include <cstring>
using namespace std;

struct Carta{
    int valor;
    string palo;
};

int main() {
    Carta carta1;

    // Ingreso de datos
    carta1.valor = 8;
    carta1.palo = "Trebol";
    Carta carta2 = {2, "Diamante"};

    //Salida de datos
    cout << carta1.valor << " " << carta1.palo << endl;
    cout << carta2.valor << " " << carta2.palo << endl;
    return 0;
}
```



```
8 Trebol
2 Diamante
```

La salida de datos, solo puede hacerse de manera individual en una estructura, campo por campo

19

Entrada/salida de valores de tipo registro

- No existe un mecanismo predefinido para la lectura o escritura de valores de tipo registro.
- El programador deberá ocuparse de la lectura/escritura de un registro, efectuando la lectura/escritura de cada uno de sus campos.

```
void leer_fecha (Fecha& f)
{
    cin >> f.dia >> f.mes >> f.anyo ;
}
```

```
void escribir_fecha (const Fecha& f)
{
    cout << f.dia << "/" << f.mes << "/" << f.anyo ;
}
```

```
struct Fecha {
    unsigned dia ;
    unsigned mes ;
    unsigned anyo ;
};
```

20

Entrada/salida de valores de tipo registro

```
#include <iostream>
using namespace std;

struct Fecha {
    unsigned dia ;
    unsigned mes ;
    unsigned anyo ;
} ;

void ingresarFecha(Fecha & fecha);
void escribirFecha(Fecha & fecha);

int main() {
    Fecha fec;
    ingresarFecha(fec);
    escribirFecha(fec);
    return 0;
}

void ingresarFecha(Fecha & fecha){
    cout << "Ingrese una fecha: ";
    cin >> fecha.dia >> fecha.mes >> fecha.anyo;
}

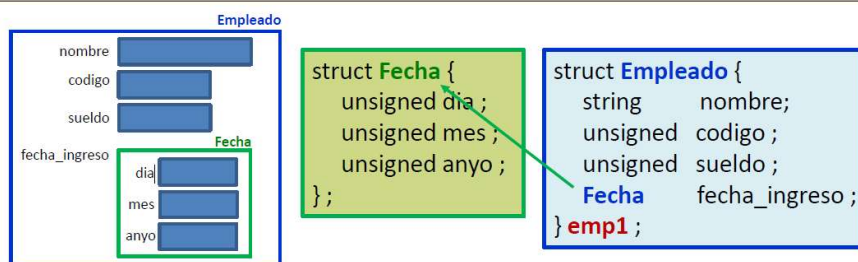
void escribirFecha(Fecha & fecha){
    cout << endl << endl << "La fecha ingresada fue: "
    << fecha.dia << "/" << fecha.mes << "/" << fecha.anyo;
}
```

Ingrese una fecha: 25 5 1810

La fecha ingresada fue: 25/5/1810

21

Registros anidados o estructuras jerárquicas



Para referirse al año de ingreso del empleado almacenado en **emp1**:

emp1.fecha_ingreso.anyo

Nombre de la variable
de tipo Empleado

Nombre del campo de la
variable de tipo Empleado

Nombre de campo,
correspondiente a la
estructura Fecha

22

Operaciones validas con estructuras

- Acceder a los campos de una estructura
- Asignar una estructura a otra estructura del mismo tipo
- Determinar el tamaño de una estructura usando el operador sizeof
- Obtener la dirección (&) de una estructura.

23

Algoritmos y Estructuras de Datos – AEDD – Asistencia 24/08

Contraseña del estudiante
jip5vz

24

Acceso a los campos de una estructura

Se puede acceder a los campos de una estructura de 2 maneras:

- Utilizando el operador punto (.)

```
struct Carta { int valor ; string palo ; } ;
Carta miCarta ;
cout << miCarta.palo ;
```

- Utilizando el operador flecha (->)

El operador flecha(->) se usa solamente cuando se tienen variables de tipo punteros a variables estructura

```
Carta miCarta ;
Carta * ptrmiCarta = &miCarta;
cout << ptrmiCarta -> palo ;
```

25

Asignaciones

Se permiten asignaciones con registros completos.

- Asignar una estructura a otra del mismo tipo:

Ejemplo: si fecha1 y fecha2 son dos variables de tipo Fecha, para almacenar cada uno de los campos de fecha1 en los campos correspondientes de fecha2, se hará:

```
fecha2 = fecha1 ;
```

La asignación es equivalente a hacer:

```
fecha2.dia = fecha1.dia ;
fecha2.mes = fecha1.mes ;
fecha2.anyo = fecha1.anyo ;
```

26

Asignaciones

Es posible asignar un valor de tipo registro, completo, a una variable o campo de otra estructura, siempre que sean del mismo tipo.

Ej: se puede asignar un registro de tipo Fecha al campo fecha_ingreso de un registro de tipo Empleado, ya que tanto el valor que se asigna como el elemento al que se asigna son del mismo tipo.

```
struct Fecha {
    unsigned dia ;
    unsigned mes ;
    unsigned anyo ;
};
```

```
struct Empleado {
    string nombre ;
    unsigned codigo ;
    unsigned sueldo ;
    Fecha fecha_ingreso ;
};
```

```
Empleado emple;
Fecha fecha2 = { 18, 10, 2001 } ;
```

```
emple.nombre = "Juan" ;
emple.codigo = 101 ;
emple.sueldo = 1000 ;
emple.fecha_ingreso = fecha2;
```

No hay ninguna otra operación disponible con registros completos. Los operadores de comparación (`==`, `!=`, `>`, `<`, `...`) no son válidos entre valores de tipo registro.

27

Tamaño de una estructura

- Es posible determinar el tamaño, en bytes, de una variable o estructura de tipo registro.
- Mediante el operador `sizeof`:

```
#include <iostream>
using namespace std;

struct persona {
    string nombre;
    string apellido;
    int edad;
    string domicilio;
};

int main() {
    persona juan;
    cout << "El tamaño de la estructura es de " << sizeof (juan)
         << " bytes";
    return 0;
}
```

El tamaño de la estructura es de 76 bytes

$24 + 24 + 4 + 24 = 76$

28

Las estructuras como elementos de arreglo

```
#include <iostream>
using namespace std;

struct Fecha {
    unsigned dia ;
    unsigned mes ;
    unsigned anyo ;
};

const int N_CITAS = 4;

int main() {
    const Fecha CUMPLEANYOS[N_CITAS] = { {1,1,2001}, {2,2,2002},
                                           {3,3,2003}, {4,4,2004} };

    cout << CUMPLEANYOS[2].mes << endl;
    return 0;
}
```

Diagrama de la estructura CUMPLEANYOS:

CUMPLEANYOS:			
1	2	3	4
1	2	3	4
2001	2002	2003	2004
0	1	2	3

Salida de la ejecución: 3

La posición en el arreglo → Campo de la estructura Arreglo

29

Estructuras que contienen arreglos

```
#include <iostream>
using namespace std;

struct Fecha {
    unsigned dia ;
    unsigned mes ;
    unsigned anyo ;
};

struct Alumno {
    string nomyape ;
    Fecha fecha_ingreso ;
    float notas[5] ;
};

int main() {
    Alumno arr_alum[2] = { {"Juan Garcia", {1,1,2002}, {10,8,10,9,8}},
                          {"Maria Viale", {12,12,2001}, {10,10,10,9,9}} };

    cout << arr_alum[0].nomyape << endl;
    cout << arr_alum[0].fecha_ingreso.dia << "/";
    cout << arr_alum[0].fecha_ingreso.mes << "/";
    cout << arr_alum[0].fecha_ingreso.anyo << endl;
    for(int i=0; i<5; i++)
        cout << arr_alum[0].notas[i] << " ";

    return 0;
}
```

Salida de la ejecución:

```
Juan Garcia
1/1/2002
10 8 10 9 8
```

30

Uso de estructuras como parámetros en funciones

Paso de Estructuras a Funciones:

- Paso de una estructura completa
- Paso de campos individuales

Por defecto, los structs se pasan por valor.

Para pasar una estructura por Valor:

- Se pasa una copia de la estructura

Para pasar una estructura por Referencia:

- Se pasa su dirección

31

Algoritmos y Estructuras de Datos – AEDD – Asistencia 24/08

Contraseña del estudiante
jip5vz

32

Ejemplo del pasaje por valor por referencia

```
#include <iostream>
#include <cstring>
using namespace std;

struct Alumno{
    string ape;
    string nom;
    int edad;
};

void CargarAlumno(Alumno & );
void MostrarAlumno(const string, const string, const int);

int main() {
    Alumno a;
    CargarAlumno(a);
    MostrarAlumno(a.ape, a.nom, a.edad);
    return 0;
}

void CargarAlumno(Alumno & alum){
    cout << "Ingrese apellido: ";
    cin >> alum.ape;
    cout << "Ingrese nombre: ";
    cin >> alum.nom;
    cout << "Ingrese edad: ";
    cin >> alum.edad;
}

void MostrarAlumno(const string ape, const string nom, const int edad){
    cout << "Apellido: " << ape << endl;
    cout << "Nombre: " << nom << endl;
    cout << "Edad: " << edad << endl;
}
```

```
Ingrese apellido: Flores
Ingrese nombre: Manuela
Ingrese edad: 25
Apellido: Flores
Nombre: Manuela
Edad: 25
```

33

Función que retorna struct

```
#include <iostream>
using namespace std;

struct Alumno{
    string ape;
    string nom;
    int edad;
    float notas[5];
};

Alumno AlumnoVacio();
Alumno CargaAlumno();
void MuestraAlumno(const Alumno a);
float PromedioNotas(const Alumno a);

int main() {
    Alumno a;
    a = AlumnoVacio();
    a = CargaAlumno();
    MuestraAlumno(a);
    return 0;
}

Alumno AlumnoVacio(){
    Alumno aux = {"", "", 0, {0}};
    return aux;
}

void MuestraAlumno(const Alumno a){
    cout << endl << endl;
    cout << "Apellido: " << a.ape << endl;
    cout << "Nombre: " << a.nom << endl;
    cout << "Edad: " << a.edad << endl;
    for (int i=0; i<5; i++){
        cout << "Nota[" << i << "] = " << a.notas[i] << endl;
    }
    cout << "Promedio Notas: " << PromedioNotas(a);
}

Alumno CargaAlumno(){
    Alumno aux;
    cout << "Ingrese apellido: ";
    cin >> aux.ape;
    cout << "Ingrese nombre: ";
    cin >> aux.nom;
    cout << "Ingrese edad: ";
    cin >> aux.edad;
    cout << "Ingrese 5 notas: " << endl;
    for (int i=0; i<5; i++){
        cout << "Nota[" << i << "] : ";
        cin >> aux.notas[i];
    }
    return aux;
}
```

34

Función que retorna struct - Continuación

```
float PromedioNotas(const Alumno a){
    int i;
    float suma = a.notas[0];
    for(i=1; i<5; i++)
        suma += a.notas[i];
    return (suma/5);
}
```

```
Ingrese apellido: Benitez
Ingrese nombre: Lucila
Ingrese edad: 20
Ingrese 5 notas:
Nota[0]: 5
Nota[1]: 5
Nota[2]: 8
Nota[3]: 7
Nota[4]: 4

Apellido: Benitez
Nombre: Lucila
Edad: 20
Nota[0] = 5
Nota[1] = 5
Nota[2] = 8
Nota[3] = 7
Nota[4] = 4
Promedio Notas: 5.8

<< El programa ha finalizado: co
<< Presione enter para cerrar es
```

35

Problema: Las notas de los estudiantes

La Facultad necesita registrar información de sus estudiantes y las materias que cursan.

Cada estudiante tiene datos personales y además debe guardar las calificaciones obtenidas en un conjunto fijo de materias.

Definir una estructura Materia que contenga: string nombre → nombre de la materia. float nota → nota final en la materia.

Definir una estructura Estudiante que contenga: int legajo → número de identificación del alumno. string nombre → nombre y apellido del estudiante. Un arreglo unidimensional de Materia con 3 materias. El programa debe:

Leer los datos de N estudiantes (N se ingresa por teclado, máximo 50).

1. Para cada estudiante, leer sus 3 materias (nombre y nota).
2. Calcular y mostrar el promedio general de cada estudiante.
3. Determinar y mostrar el estudiante con el mejor promedio.

36

Algoritmos y Estructuras de Datos – AEDD – Agenda 24/08

- ✓ Arreglos Multidimensionales.
- ✓ Inicialización de arreglos bidimensionales.
- ✓ Acceso a los elementos de la matriz.
- ✓ Pasaje de matriz como parámetro
- ✓ Definición de Estructura.
- ✓ Declaración de variables de estructura.
- ✓ Inicialización de variables de estructura.
- ✓ Acceso a los elementos de la estructura.
- ✓ Las estructuras como elemento de arreglos.
- ✓ Estructuras que contiene arreglos.
- ✓ Estructuras como parámetros de funciones.
- ✓ Ejemplo del pasaje por valor y por referencia.
- ✓ Función que retorna Struct

¡¡Objetivo cumplido!!!

37

AEDD – Otras tareas !!!!!

Capítulo 13 - “Estructuras”

Libro: “C++ para ingeniería y ciencias” 2da edición - Gary J.
Bronson, pág. 711

Capítulo 6 - “Tipos Compuestos”

Libro: Benjumea-Roldan, pág. 64
Universidad de Málaga

“Estructuras” - Página 543

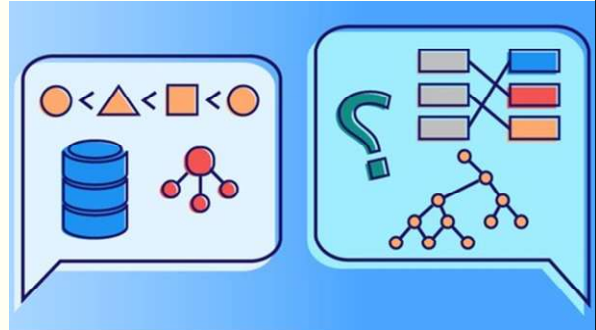
Libro: “Fundamentos de la programación” - Luis Hernández
Yáñez - Universidad Complutense

38

ALGORITMOS Y ESTRUCTURA DE DATOS

Comisión F – 2026 – Cursado Anual

¿DUDAS?

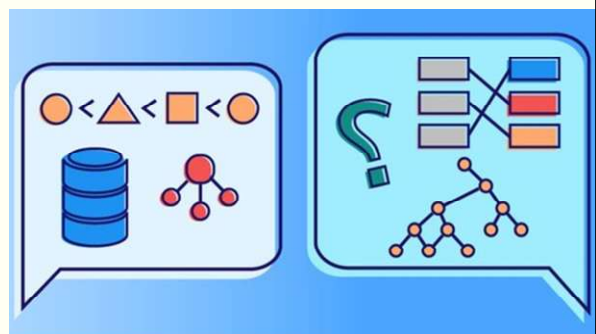


39

ALGORITMOS Y ESTRUCTURA DE DATOS

Comisión F – 2025 – Cursado Anual

¡Muy buen fin de semana!



40